

High-Level Design Document

Product: QuantTrade Life

Domain: AI-Powered Gamified Financial Literacy Platform

Version: 1.0

1. Executive Summary

QuantTrade Life is a **character-driven financial simulation platform** designed to improve financial literacy across multiple age cohorts through **story gameplay, AI coaching, market simulation, behavioral finance feedback, and community participation**.

The platform allows users to create a character, progress through life stages, earn and allocate income, invest across financial instruments, respond to market and life events, and build long-term wealth. The product combines:

- life simulation
- financial education
- gamification
- AI mentoring
- community interaction
- progressive learning paths

The system is architected to support:

- beginner to advanced users
- real and simulated finance scenarios
- long-term retention loops
- scalable AI personalization
- moderation and community governance

2. Product Goals

Primary Goals

- Improve financial literacy through applied simulation
- Build long-term user engagement through progression and narrative
- Personalize learning through AI-driven coaching
- Help users understand investing, diversification, risk, and wealth-building
- Create a safe environment for learning finance without real-money exposure at MVP stage

Secondary Goals

- Build a defensible engagement moat via behavioral finance profiles
- Create community-led strategy sharing and peer learning
- Prepare platform for future premium monetization

- Create a bridge from simulated learning to real-world financial action later

3. Non-Goals for MVP

The MVP should **not** attempt to do the following:

- Real-money brokerage execution
- Options/futures/leveraged derivatives trading
- Social copy trading with live capital
- Full open-world 3D game complexity
- Full blockchain wallet integration
- Enterprise financial planning workflows
- Tax filing or regulated financial advice

Strong recommendation: keep the first release sharply focused on **learning efficacy and retention**.

4. User Segments

Segment A: Student

- Age group: under 18 or entry learners
- Needs: money basics, earning, saving, spending, simple investing concepts
- Product mode: guided narrative, highly visual, low complexity

Segment B: College / Young Adult

- Age group: 18–25
- Needs: budgeting, first paycheck, debt awareness, first investments
- Product mode: structured progression, trade-offs, life choices, basic ETF's, compounding

Segment C: Early Career

- Age group: 25–35
- Needs: portfolio growth, diversification, ETFs, compounding, basic analysis, IPO, Strategy Trading
- Product mode: more autonomy, deeper financial tools

Segment D: Family / Stability

- Age group: 35–45
- Needs: risk management, dependents, long-term planning, allocation discipline, IPO, Family financial planning,

Segment E: Mastery

- Age group: 45+
- Needs: wealth optimization, advanced allocation, capital preservation, scenario analysis, Retirement planning

5. Core Product Principles

1. **Learn by doing, not by reading**
2. **Character progression should mirror financial maturity**
3. **AI should explain, not merely recommend**
4. **The system must reward discipline, not reckless speculation**
5. **Community must be safe, moderated, and age-sensitive**
6. **Finance should feel experiential, not academic**
7. **Progressive complexity is mandatory**

6. High-Level Functional Scope

Core Modules

- Identity and onboarding
- Character progression engine
- Wallet and portfolio engine
- Market simulation engine
- Scenario and story engine
- AI mentor and behavioral engine
- Learning and curriculum engine
- Community and moderation engine
- Gamification and achievement engine
- Admin and analytics console

7. System Context Diagram

At a high level, the system consists of:

- **Client applications**
 - web app
 - responsive mobile web / PWA
 - optional future native mobile app
- **Application backend**
 - user management
 - gameplay orchestration
 - wallet/portfolio logic
 - scenarios
 - community
 - achievements
 - analytics
- **AI services**
 - mentor chatbot
 - behavioral analysis
 - recommendation engine
 - moderation bot
 - retrieval system

- **Data layer**
 - relational database
 - vector database capability via pgvector
 - cache
 - event logs
 - market data storage
- **External integrations**
 - market data providers
 - news providers
 - LLM providers
 - notification providers

8. Architecture Overview

Recommended Architecture Style

Use a **modular monolith for MVP**, designed with clean domain boundaries so it can later evolve into services.

This is the correct move. A premature microservices strategy here would be operationally inefficient.

Major Layers

- Presentation Layer
- API Layer
- Domain/Application Layer
- AI Orchestration Layer
- Data Access Layer
- Infrastructure Layer

9. Frontend Architecture

Recommended Stack

- Next.js
- TypeScript
- Tailwind CSS
- React Query
- Zustand
- React Hook Form
- React Three Fiber / Three.js for controlled 3D elements
- Framer Motion for transitions
- Remotion
- Unreal Engine if necessary

Frontend Modules

- Authentication and onboarding - auth take from login which we already have - just add DOB, Gender - Add in DB - based on which auto select

the character in game

- Dashboard
- Animated Apple Wallet with Credit card and portfolio
- Investment/trading screen engine
- Scenario engine UI
- Learning quests UI
- Community hub
- Profile and achievements
- Admin moderation panel

Frontend Design Principles

- dark, premium, institutional, medieval age aesthetic
- gamified but credible
- highly visual progression
- strong information hierarchy
- avoid cartoonish cheap fintech look
- accessible for beginners

10. Backend Architecture

Recommended Stack

- FastAPI
- Python 3.11+
- SQLAlchemy
- Pydantic
- PostgreSQL
- pgvector
- Redis
- Celery or background task queue
- WebSocket support for community and event updates

Backend Domains

1. Auth Domain

- session/token handling
- role management

2. User Profile Domain

- demographic information
- experience level
- learning goals
- preferences
- risk comfort level

3. Character Domain

- avatar metadata
- life stage

- character progression
- financial traits
- behavioral profile

4. Wallet Domain

- virtual cash balance
- salary income
- cash flow entries
- expenses
- rewards

5. Portfolio Domain

- holdings
- trades
- allocations
- pnl
- diversification score
- historical snapshots

6. Market Domain

- instrument master
- prices
- time series
- news tags
- sector classifications
- volatility profiles

7. Scenario Domain

- life events
- market events
- mission generation
- branching outcomes

8. Education Domain

- quests
- lessons
- knowledge checkpoints
- mastery levels

9. Community Domain

- channels
- posts
- comments
- reports
- moderation actions

10. AI Domain

- mentor interactions

- recommendations
- behavior analytics
- moderation classifier
- retrieval context builder

11. Gamification Domain

- badges
- streaks
- XP
- missions
- level progression
- leaderboards

12. Analytics Domain

- learning progress
- engagement telemetry
- retention metrics
- scenario completion
- portfolio improvement metrics

11. Core Domain Flows

A. Onboarding Flow

1. User signs up - if not signed up
2. User get learning stage
3. User gets avatar/character
4. System asks baseline finance questions
5. AI assigns initial literacy profile
6. System configures starting storyline, Apple wallet, and quest path

B. Daily Gameplay Flow

1. User logs in
2. Animated loading screen of medieval age aesthetic
3. Dashboard shows financial status and missions
4. User receives life or market event
5. User makes decisions
6. Portfolio/wallet updates
7. AI explains outcome
8. XP // coins, achievements, and progression update

C. Investment Flow

1. User views available instruments
2. User researches with AI guidance
3. User buys/sells simulated asset
4. Trade recorded
5. Portfolio recalculated

6. Risk/diversification scores refreshed
7. Behavioral engine tags decision pattern

D. Community Flow

1. User enters stage-specific or general community
2. User posts or replies
3. Content passes moderation pipeline
4. AI bot assists and enforces policy
5. Users engage via likes, comments, strategy discussions

12. Gameplay and Story Engine Design

Character-Centric Model

The character is the primary object of progression, not a city or static portfolio.

Character Attributes

- age/stage
- experience level
- financial confidence
- discipline score
- emotional resilience
- savings habit
- risk profile
- learning progress
- social reputation in community

Story Logic

The system should deliver semi-structured narrative arcs:

- first paycheck
- debt pressure
- War scenarios
- market boom
- recession
- emergency expense
- opportunity cost decision
- family responsibility
- entrepreneurial opportunity

Story Engine Approach

Use a hybrid system:

- deterministic rules for key economics
- template-based narrative generation
- AI expansion for dynamic personalization

This is crucial. Do not let the LLM own core game state. The LLM should narrate and explain, not be the source of truth for economics.

13. Financial Simulation Engine

Financial Components

- cash balance
- salary income
- emergency fund
- expenses
- holdings
- dividends
- investment returns
- liabilities later phase
- savings rate

Asset Universe for MVP

- blue-chip stocks
- growth stocks
- ETFs
- bonds
- cash
- crypto
- REITs
- IPOs
- commodities
- private investments
- options sandbox

Portfolio Logic

- weighted allocation
- expected return model
- volatility model
- concentration risk
- diversification score
- scenario stress response

Simulation Inputs

- historical price data
- synthetic data for branching scenarios
- macro scenario rules
- user actions
- life-event modifiers

14. AI System Design

AI Components

A. AI Mentor

Responsibilities:

- explain concepts
- answer finance questions
- explain scenario outcomes
- give tailored learning guidance
- simplify concepts based on stage

Inputs:

- user profile
- character state
- portfolio state
- recent actions
- knowledge level
- relevant retrieval context

Outputs:

- conversational response
- learning recommendation
- warning or reinforcement
- next best action guidance

B. Behavioral Finance Engine

Responsibilities:

- classify user behavior patterns
- identify panic selling
- identify overtrading
- detect concentration risk behavior
- detect impulsive behavior

Inputs:

- trade history
- reaction timing
- asset concentration
- event-driven decisions
- session behavior

Outputs:

- behavioral traits
- coaching interventions
- risk nudges
- progression score adjustments

C. Scenario Generator

Responsibilities:

- select relevant life events
- schedule market conditions
- personalize difficulty
- generate decision trees

Inputs:

- stage
- knowledge level
- recent performance
- engagement signals

Outputs:

- event packages
- mission content
- branching options

D. Community Bot

Responsibilities:

- answer common questions
- route users to correct stage spaces
- flag misinformation
- de-escalate toxicity
- summarize community discussions

E. Recommendation Engine

Responsibilities:

- suggest content/quests
- suggest diversified portfolio learning exercises
- recommend community groups
- recommend scenario intensity

15. AI Guardrails

This system deals with finance and potentially minors. Guardrails must be explicit.

Guardrails

- no direct real-money advice in MVP
- clear simulation-only disclaimers
- age-appropriate content filtering
- moderation against scams, pump-and-dump behavior, and harmful content
- separate educational guidance from individualized financial advice
- human review queue for sensitive community escalations

AI Control Principle

The AI can:

- explain
- teach
- simulate
- coach

The AI cannot:

- act as a licensed advisor
- promise returns

- encourage reckless speculation
- produce unsafe community moderation decisions without auditability

16. Community Architecture

Community Types

- General financial literacy
- Stage-based channels
 - student
 - college
 - early career
 - family
 - mastery
- topic channels
 - budgeting
 - investing basics
 - ETFs
 - market psychology
 - AI in finance

Community Features

- posts
- comments
- likes/upvotes
- strategy threads
- stage badges
- moderation reporting
- AI assistant responses
- community challenges

Moderation Pipeline

1. User submits content
2. Rule-based filters run
3. AI moderation classifier scores content
4. High-risk content routed to hold/review
5. Approved content published
6. Repeat offenders scored

Community Bot Actions

- remind users of channel rules
- answer FAQs
- suggest relevant learning resources
- recommend moving age-inappropriate discussions to proper channels

17. Gamification Architecture

Engagement Systems

- XP
- levels
- streaks
- badges
- milestones
- quests
- leaderboards
- community reputation

Reward Types

- unlock new scenarios
- unlock new avatar accessories
- unlock advanced analytics
- unlock community badges
- unlock new learning domains

Daily/Weekly Loops

- daily mission
- portfolio health review
- scenario challenge
- community contribution challenge
- streak reward

18. Data Architecture

Primary Database

PostgreSQL as system of record

Secondary Support

- pgvector for embeddings
- Redis for caching/session/temp state
- object storage for media assets and 3D assets
- analytics/event store for product telemetry

Core Entity Groups

- users
- characters
- wallets
- transactions
- portfolios
- holdings
- assets
- price_history
- scenarios
- scenario_runs

- quests
- lessons
- ai_conversations
- behavior_signals
- community_posts
- community_comments
- moderation_flags
- achievements
- notifications

19. Suggested Logical Data Model

UserProfile

- user_id
- stage_preference
- finance_experience
- goals
- risk_tolerance
- age_band

Character

- character_id
- user_id
- avatar_config
- current_stage
- xp
- level
- discipline_score
- behavior_profile_id

Wallet

- wallet_id
- character_id
- cash_balance
- total_earned
- total_spent
- total_invested

Transaction

- transaction_id
- wallet_id
- type
- amount
- source

- timestamp

Portfolio

- portfolio_id
- character_id
- total_value
- pnl
- diversification_score
- risk_score

Holding

- holding_id
- portfolio_id
- asset_id
- quantity
- avg_cost
- current_price
- allocation_pct

Asset

- asset_id
- symbol
- name
- asset_type
- sector
- risk_band

PriceHistory

- asset_id
- timestamp
- open
- high
- low
- close
- volume

Scenario

- scenario_id
- title
- stage
- category
- difficulty
- template_data

ScenarioRun

- run_id
- character_id

- scenario_id
- decisions
- outcome
- completed_at

Quest

- quest_id
- stage
- title
- reward_xp
- reward_currency
- unlock_condition

AIConversation

- conversation_id
- user_id
- context_type
- input_text
- response_text
- model_used
- created_at

BehaviorSignal

- signal_id
- character_id
- signal_type
- score
- evidence_json
- created_at

CommunityPost

- post_id
- user_id
- channel_id
- content
- status
- created_at

ModerationFlag

- flag_id
- entity_type
- entity_id
- risk_score
- reason
- action_taken

20. API Architecture

API Style

REST for most platform operations, with WebSockets for real-time features.

Example API Groups

Auth APIs

- POST /auth/register
- POST /auth/login
- POST /auth/logout

Character APIs

- GET /character/me
- POST /character/create
- PATCH /character/update

Wallet APIs

- GET /wallet/me
- GET /wallet/transactions

Portfolio APIs

- GET /portfolio/me
- GET /portfolio/holdings
- POST /portfolio/trade
- GET /portfolio/performance

Market APIs

- GET /market/assets
- GET /market/quote/{symbol}
- GET /market/history/{symbol}

Scenario APIs

- GET /scenarios/current
- POST /scenarios/{id}/decision
- GET /scenarios/history

Quest APIs

- GET /quests/daily
- POST /quests/{id}/complete

Community APIs

- GET /community/channels
- GET /community/feed
- POST /community/posts
- POST /community/comments
- POST /community/report

AI APIs

- POST /ai/mentor/chat
- GET /ai/behavior/profile

- GET /ai/recommendations

Admin APIs

- GET /admin/moderation/queue
- POST /admin/moderation/action

21. Event-Driven Design

Even in a modular monolith, key system operations should emit domain events.

Important Events

- user_registered
- character_created
- quest_completed
- trade_executed
- scenario_started
- scenario_completed
- behavior_signal_generated
- post_created
- moderation_flagged
- achievement_unlocked

Why This Matters

This allows:

- auditability
- future service decomposition
- analytics enrichment
- notification triggers
- AI workflow hooks

22. Real-Time Features

Use WebSockets selectively for:

- live community updates
- live market changes
- mission notifications
- scenario prompts
- moderation status feedback in community tools later

Do not over-engineer real-time in MVP. Real-time should serve engagement, not complexity theater.

23. Security Design

Security Requirements

- \RBAC for admin/mod/community roles
- rate limiting for AI and community endpoints
- audit logs for moderation/admin actions

- PII minimization
- encrypted secrets management
- secure third-party integration handling

Finance-Specific Safety

- separate simulated balances from any future real-money systems
- prevent users from interpreting simulation outputs as fiduciary advice
- policy-driven model outputs for investment guidance

24. Scalability Approach

MVP

- modular monolith
- single relational DB
- Redis cache
- async background jobs
- one AI orchestration pipeline

Growth Phase

Split domains if load demands it:

- AI service
- community service
- analytics pipeline
- market data ingestion service

High-Scale Phase

- dedicated event bus
- search indexing
- feature store for behavior models
- recommendation service
- moderation review operations

25. Observability

Monitoring Requirements

- API latency
- error rates
- background job failures
- AI response latency
- moderation queue depth
- market data freshness
- user funnel drop-off
- quest completion rates

Logging

- structured backend logs
- moderation decision logs

- AI prompt/response metadata logs with privacy controls
- scenario engine audit trail

Analytics

Track:

- D1/D7/D30 retention
- average session length
- daily quest completion
- AI mentor usage
- progression by stage
- community participation
- literacy uplift proxy metrics

26. Content Management and Admin Tools

Admin Capabilities

- manage scenarios
- manage quests and curriculum
- review flagged community content
- tune reward systems
- view analytics dashboards
- manage AI prompt templates
- configure stage rules
- manage asset universe and educational mappings

A strong internal admin tool is not optional. Without this, content iteration becomes operationally expensive.

27. 3D / Experience Layer Design

Recommendation

Use 3D sparingly and strategically.

Do:

- avatar showcase
- financial growth visual states
- premium transitions
- scenario scene intros

Do not:

- build a full 3D explorable world for MVP
- make gameplay dependent on heavy graphics

Visual State Concept

Character environment evolves based on financial health:

- unstable finances → worn environment
- disciplined investing → improved lifestyle visuals

- milestone achievements → richer scene state

This creates emotional payoff without destroying delivery velocity.

28. MVP Scope Recommendation

MVP Must Include

- onboarding
- character creation
- wallet
- basic investment simulation
- student and college stages
- AI mentor
- daily missions
- simple scenarios
- community channels
- AI moderation bot
- dashboard and portfolio view
- achievements and streaks

MVP Should Exclude

- mastery stage complexity
- advanced technical indicators
- Monte Carlo deep simulation
- options trading
- open strategy marketplace
- full real-money bridge
- heavy multiplayer features

This prioritization is essential. Otherwise the project will become bloated and stall.

29. Future Roadmap

Phase 2

- early career and family stages
- more scenario complexity
- richer behavior profiling
- expanded community mechanics
- better leaderboards
- AI-generated adaptive quests

Phase 3

- mastery stage
- strategy marketplace
- creator economy layer
- real brokerage integrations

- regulated transition pathways
- advanced analytics and scenario labs

30. Success Metrics

Learning Metrics

- improvement in diversification behavior
- reduction in panic-selling behavior in simulations
- knowledge checkpoint pass rates
- portfolio quality improvement over time

Product Metrics

- D7 and D30 retention
- daily active users
- average weekly missions completed
- AI mentor weekly active usage
- community contribution rate

Monetization Metrics Later

- free-to-paid conversion
- premium feature engagement
- community subscription lift

31. Key Risks and Mitigations

Risk 1: Product becomes too broad

Mitigation:

- lock MVP scope
- focus on first two stages only

Risk 2: AI gives poor or unsafe finance guidance

Mitigation:

- strong prompt constraints
- retrieval-backed explanations
- educational framing only

Risk 3: Community becomes noisy or unsafe

Mitigation:

- stage-based segmentation
- AI and human moderation
- safety rules and reputation system

Risk 4: Gamification becomes shallow

Mitigation:

- tie progression to real financial behaviors
- reward discipline, not random activity

Risk 5: 3D scope explodes

Mitigation:

- decorative and state-based 3D only in MVP

32. Final Architecture Recommendation

The correct implementation strategy is:

- **Next.js frontend**
- **FastAPI modular monolith backend**
- **PostgreSQL + pgvector**
- **Redis cache + background workers**
- **AI mentor + behavior engine + moderation bot**
- **character-based progression**
- **student and college focused MVP**
- **community with stage-based segmentation**
- **3D as enhancement, not core dependency**

That is the most capital-efficient and execution-efficient architecture.

33. Closing Summary

QuantTrade Life should be built as a **financial behavior simulator**, not just a finance game.

The differentiator is not charts, avatars, or fake trading.

The differentiator is this stack:

- character identity
- applied financial decision-making
- AI explanation
- behavioral finance feedback
- narrative progression
- moderated peer learning

That combination is materially stronger than a standard edtech or fintech app.

.